

Réalisation

5.1 Introduction

Cette section présente la concrétisation du système Smart School IoT à travers les interfaces effectivement développées et la simulation du circuit électronique. Les captures d'écran qui suivent illustrent le fonctionnement réel de l'application : page d'accueil, tableau de bord en temps réel, supervision détaillée d'une salle, mécanisme d'alertes, contrôle distant de la serrure, ainsi que la simulation du circuit ESP32 sur la plateforme Wokwi. Chaque figure est accompagnée d'une description fonctionnelle précise.

5.2 Page d'accueil

La page d'accueil constitue le point d'entrée visuel de l'application. Elle a été conçue selon une approche moderne en *glassmorphism*, avec une image de fond animée et un panneau translucide flou qui met en valeur l'identité du projet. Le bouton principal redirige l'utilisateur vers le tableau de bord opérationnel. Trois piliers du système sont mis en avant en pied de panneau : **Surveillance IoT**, **Alertes en Temps Réel** et **Gestion Éco-Énergétique**.

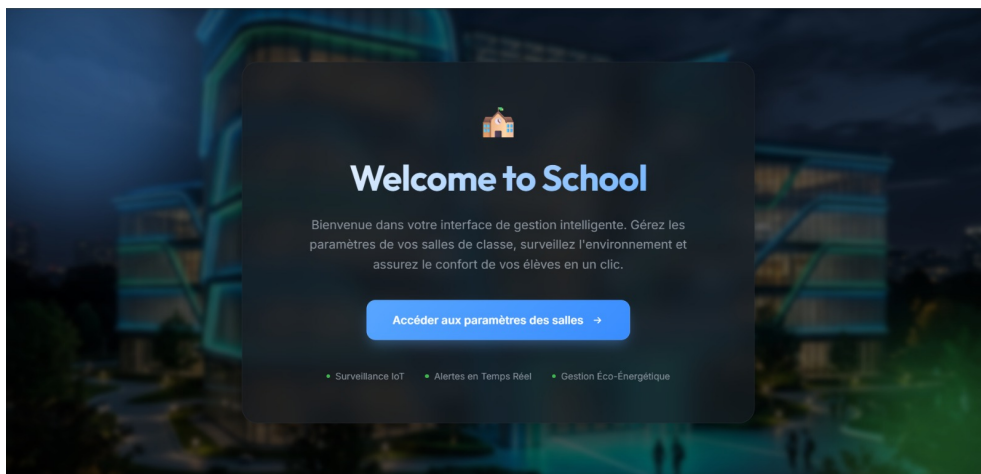


Figure 5.1 -- Page d'accueil de l'application Smart School Dashboard.

5.3 Tableau de bord principal

Le tableau de bord est l'interface centrale de supervision. Il regroupe, dans une vue unique, les indicateurs globaux (KPI) en haut de l'écran et l'état temps réel de chaque salle de classe. Les KPI affichent le nombre de salles actives, la température moyenne, le nombre d'alertes en cours et le compteur cumulé des élèves détectés. Chaque carte de salle expose les six mesures principales -- température, humidité, luminosité, niveau sonore, qualité de l'air et présence -- ainsi qu'un bouton de verrouillage et un bouton d'alerte.

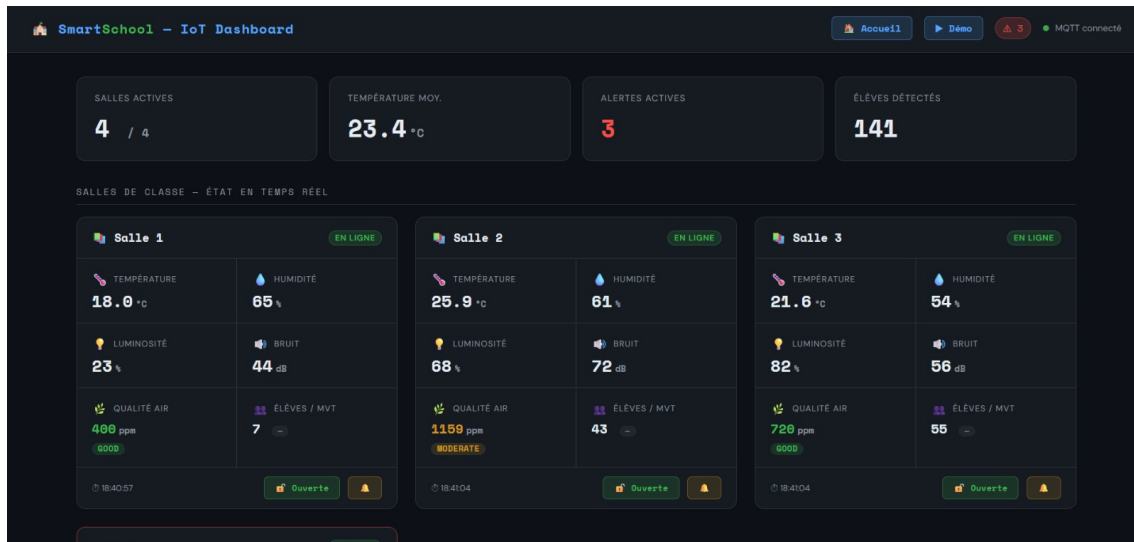


Figure 5.2 -- Vue d'ensemble du tableau de bord avec les indicateurs globaux et trois salles supervisées simultanément.

5.4 Carte d'une salle : état normal, détection de mouvement et verrouillage

La carte d'une salle synthétise l'ensemble des mesures issues des capteurs associés. Lorsqu'un mouvement est détecté par le capteur PIR, l'indicateur *Élèves / MVT* bascule visuellement sur un badge rouge *MVT* pour signaler une activité instantanée. Le bouton *Ouverte / Verrouillée* reflète l'état physique du relais côté ESP32. Cliquer sur ce bouton publie une commande MQTT ; l'ESP32, abonné à ce topic, applique l'action et publie en retour un accusé de réception.



Figure 5.3 -- Carte de la Salle 1 en fonctionnement normal : mouvement détecté (badge MVT) et serrure ouverte.



Figure 5.4 -- Carte de la Salle 1 après commande de verrouillage : badge rouge Verrouillée et retour visuel immédiat.

5.5 Système d'alertes en temps réel

Le moteur d'alertes du serveur Node.js compare en continu chaque mesure publiée aux seuils configurables. Trois niveaux de gravité sont définis : **INFO**, **WARNING** et **CRITICAL**. Chaque alerte est horodatée, persistée dans *history.json* et diffusée via Socket.IO à tous les clients connectés. Lorsqu'une valeur revient sous son seuil pendant une durée minimum, l'alerte correspondante disparaît automatiquement de la liste active (*auto-effacement*). Le journal MQTT, en bas du tableau de bord, donne un accès direct au flux brut des messages publiés sur le broker, utile pour la vérification technique en direct.

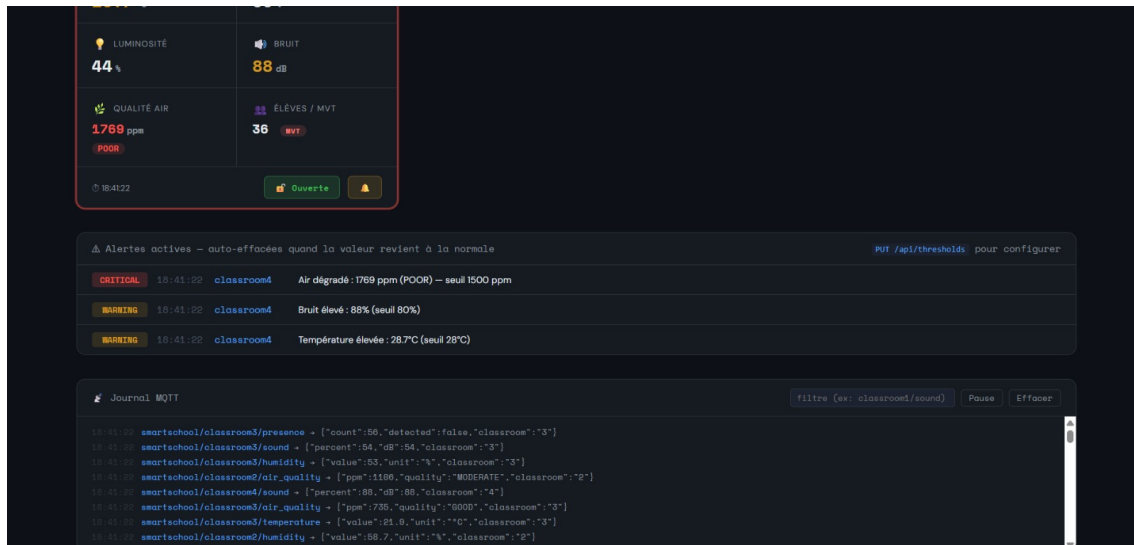


Figure 5.5 -- Trois alertes simultanées sur la Salle 4 (CRITICAL air, WARNING bruit, WARNING température) et journal MQTT en direct.

5.6 Test de propagation des seuils via le capteur DHT22

Pour valider la chaîne complète *capteur* → *ESP32* → *MQTT* → *serveur* → *dashboard*, nous avons utilisé l'éditeur de capteurs intégré à Wokwi pour modifier la valeur du DHT22 en direct. La température a été poussée à 55,4°C via le curseur, dépassant largement le seuil de 28°C ; la nouvelle valeur s'est propagée sur le tableau de bord en moins d'une seconde, déclenchant immédiatement une alerte WARNING horodatée dans le journal d'alertes.

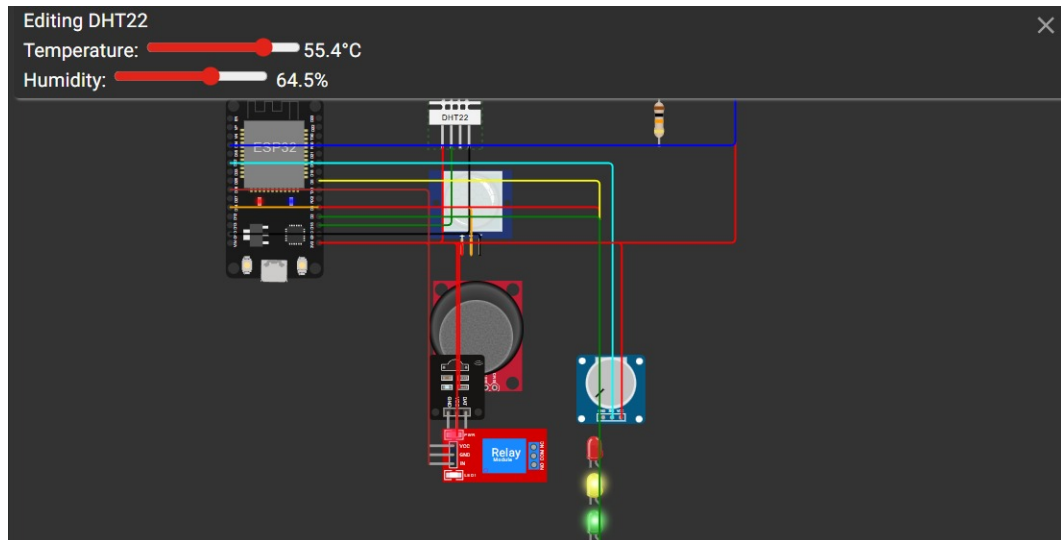


Figure 5.6 -- Manipulation du capteur DHT22 dans Wokwi : réglage manuel de la température à 55,4°C et de l'humidité à 64,5%.



Figure 5.7 -- Propagation immédiate de la mesure côté dashboard : la Salle 1 affiche 55,4°C en orange (dépassement du seuil).



Figure 5.8 -- Alerte générée automatiquement et inscrite dans le journal : Température élevée : 55,4°C (seuil 28°C).

5.7 Simulation du circuit ESP32 sur Wokwi

L'ensemble du circuit a été modélisé dans Wokwi à partir du fichier *diagram.json*. Le simulateur reproduit fidèlement le comportement physique de l'ESP32 et de ses périphériques : DHT22 (température/humidité), capteur PIR, photorésistance LDR, joystick (proxy bruit), MQ-135 (proxy qualité d'air), récepteur infrarouge (présence), module relais (serrure) et trois LEDs de signalisation. Chaque composant interactif peut être manipulé en direct, ce qui permet de tester les règles autonomes embarquées et la chaîne MQTT sans matériel physique.

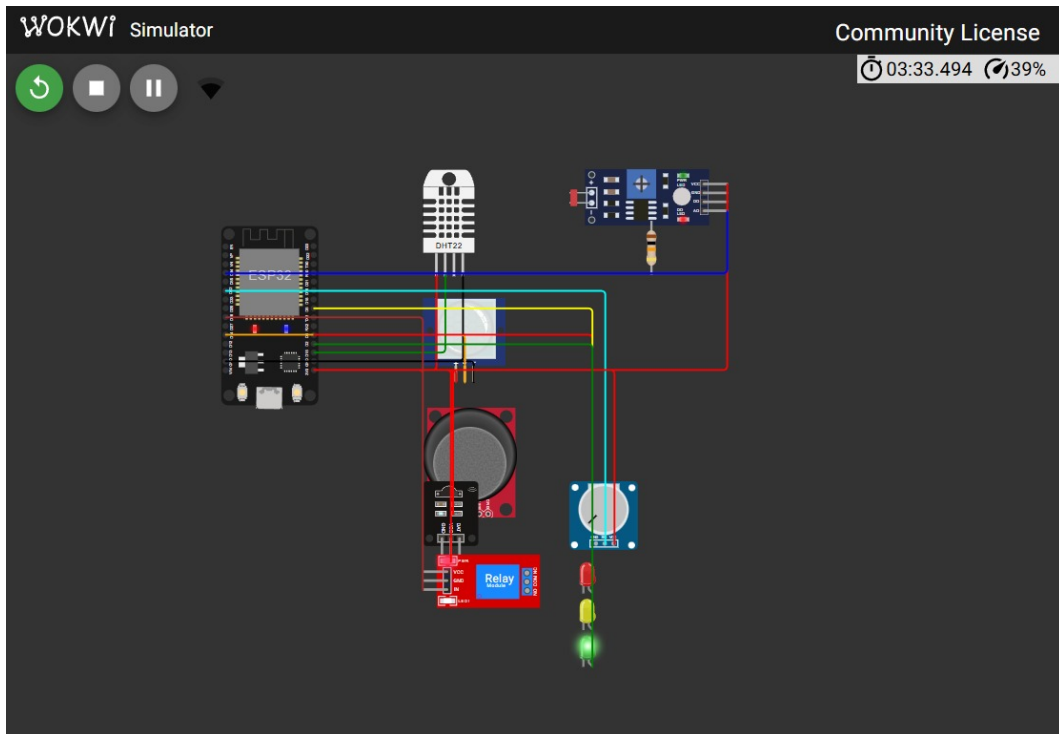


Figure 5.9 -- Simulateur Wokwi avec barre d'outils (lecture, arrêt, pause) et licence communautaire active.

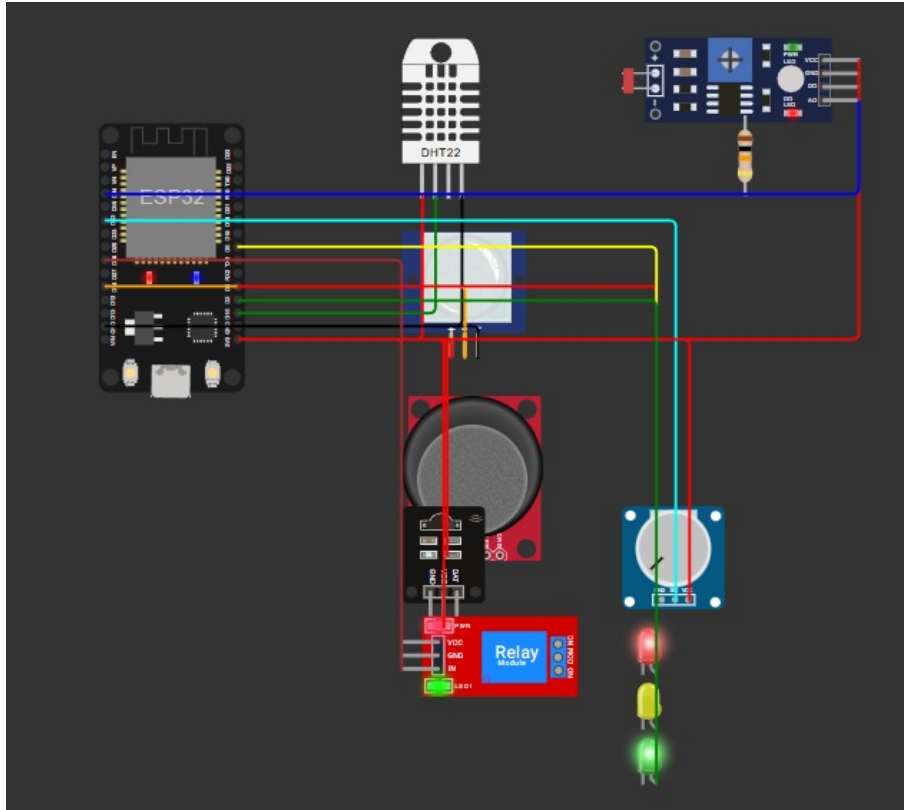


Figure 5.10 -- Vue rapprochée du circuit : ESP32 au centre, DHT22 en haut, MQ-135, joystick, potentiomètre, module relais et LEDs de signalisation.

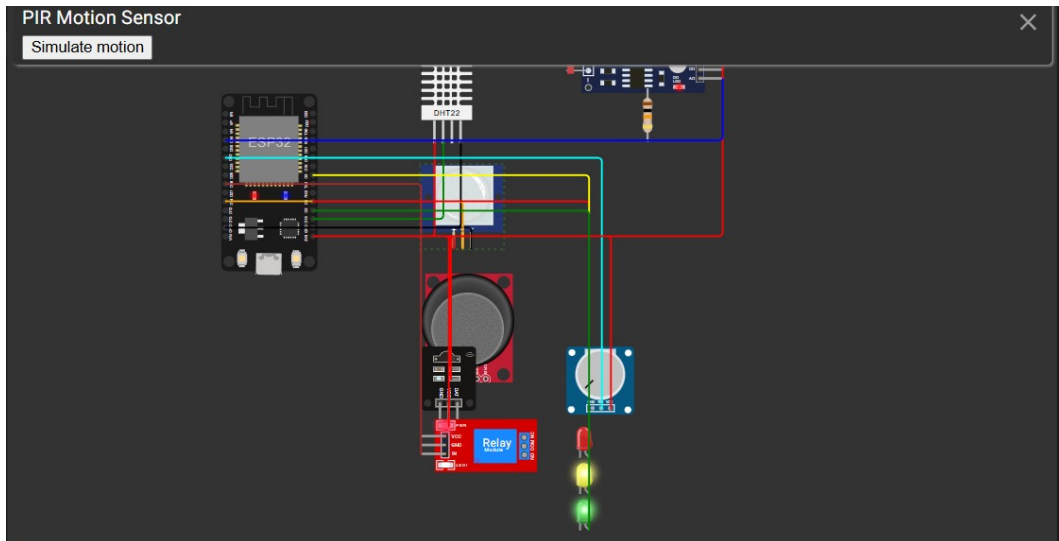


Figure 5.11 -- Manipulation du capteur PIR dans Wokwi via le bouton Simulate motion.

5.8 Conclusion

La phase de réalisation a permis de concrétiser l'ensemble des spécifications fonctionnelles définies dans les chapitres précédents. Le dashboard développé est **opérationnel, ergonomique et réactif**, et offre une vue consolidée sur quatre salles simultanément. Les manipulations effectuées via Wokwi (DHT22, PIR, relais) confirment que la chaîne complète capteur → ESP32 → broker MQTT → serveur Node.js →

dashboard fonctionne en temps réel avec une latence inférieure à une seconde, et que le moteur d'alertes réagit fidèlement aux dépassements de seuils.